

AI 활용 기술 문서

탄환 회수자 · Bullet Reclaimer | 제작 과정의 AI 활용 범위, 사람의 검토와 외부 의존성 기록

활용 원칙

AI는 게임 기획을 구체화하고, 코드 구조·충돌 로직·테스트 항목을 제안하며, 구현 결과를 점검하는 제작 보조 도구로 활용했다. 게임 플레이 중 외부 생성형 AI API를 호출하지 않는다. 따라서 심사자는 네트워크 키나 유료 계정 없이 정적 웹 빌드만으로 동일하게 플레이할 수 있다.

AI 도구 및 적용 범위

도구	활용 내용	사람의 검토·결정
OpenAI Codex	공모전 조건 분석, 웹 기술 스택 비교, Phaser + TypeScript + Vite 초기 구조 제안	웹 제출과 GitHub Pages 배포를 선택하고, 런타임 AI/API는 사용하지 않기로 결정
OpenAI Codex	단일 탄환, 시간 정지 조준, 도탄 예상선, 회수 루프의 구현 보조	플레이어도 자신의 탄환에 즉사하는 규칙을 핵심 긴장 요소로 확정
OpenAI Codex	선분-원 연속 충돌, 경로 탐색, 데이터형 스테이지, Web Audio 효과음 모듈 제안	일반 적은 즉사, 보스만 3중 보호막으로 차별화하고 테스트·브라우저 검증 후 반영

핵심 지시와 산출물

목적	대표 프롬프트/지시 요약	적용 결과
게임 루프	브라우저에서 즉시 실행되는 tap다운 액션. 총알은 한 발이며, 조준 중 시간 정지와 반사 경로를 보여 주고 발사 뒤 직접 회수한다.	조준·발사·정지·회수 상태를 분리한 게임 루프 구현
전투 안정성	빠른 탄환이 대상을 통과하지 않도록 연속 충돌 판정과 회귀 테스트를 제안한다.	선분-원 교차 기반 충돌 처리와 단위 테스트 적용
콘텐츠·연출	5개 구역과 보스, 코드 기반 픽셀 캐릭터·파티클·효과음을 외부 에셋 없이 구성한다.	데이터 기반 5스테이지·코어 보스·Web Audio 기반 효과음 적용
제출 검증	공개 GitHub Pages에서 키보드 입력, 한국어 UI, 최신 배포 상태를 확인한다.	타입 검사, 단위 테스트, 프로덕션 빌드, 브라우저 콘솔·입력 검증 수행

구조, 검증, 출처

AI 제안이 반영된 구조

영역	구현
게임 상태	조준/발사/회수 상태를 분리하여 시간 정지와 위험 구간을 명확히 처리
충돌	선분-원 교차 판정으로 고속 탄환의 누락을 줄이고, 반사 횟수를 제한
스테이지	적·장애물·보스 정보를 데이터로 정의해 5개 구역을 확장 가능하게 구성
오디오·아트	Phaser Graphics와 Web Audio API로 픽셀풍 도형, 파티클, 효과음을 코드 생성
검증	TypeScript 검사, 충돌·경로 탐색 단위 테스트, Vite 프로덕션 빌드와 브라우저 수동 점검

외부 에셋 및 오픈소스 출처

구분	출처 / 라이선스	사용 범위
이미지·사운드 에셋	외부 에셋 미사용	캐릭터, 보스, 장애물, 파티클은 코드로 렌더링 / 효과음은 Web Audio로 합성
Phaser 3.90.0	phaser.io · MIT License	2D 렌더링, 씬, 입력, 물리 보조
Vite 7.3.5	vite.dev · MIT License	개발 서버와 정적 프로덕션 빌드
TypeScript 5.9.3	typescriptlang.org · Apache-2.0	게임 로직의 정적 타입 검사

최종 확인 사항

AI가 제안한 결과는 그대로 자동 채택하지 않았다. 게임 규칙, 난이도, 보스 체력, 외부 AI 미사용 여부, 배포 방식은 개발자가 최종 결정했다. 구현 후에는 TypeScript 검사, 충돌·경로 탐색 테스트, Vite 빌드, 공개 브라우저 실행을 통해 오류 여부를 확인했다.

상세 작업 로그: 저장소의 docs/ai-usage-log.md에 날짜별 도구, 지시, 채택 결과와 수정 판단을 기록한다.